Semana 17 — Conteúdo de 2 aulas

Computação em Nuvem e Virtualização

Ricardo Pasquati Pontarolli

Técnico em Redes de Computadores · IFSP Boituva · pasquati@ifsp.edu.br · v44

Semana 17 — Conteúdo de 2 aulas

- **Aula 27:** Modelos de Gerenciamento em Nuvem
- **Aula 28:** Contêineres: Conceitos e VM $\times$ Contêiner

Aula 27 — Modelos de Gerenciamento em Nuvem

Objetivos da Aula

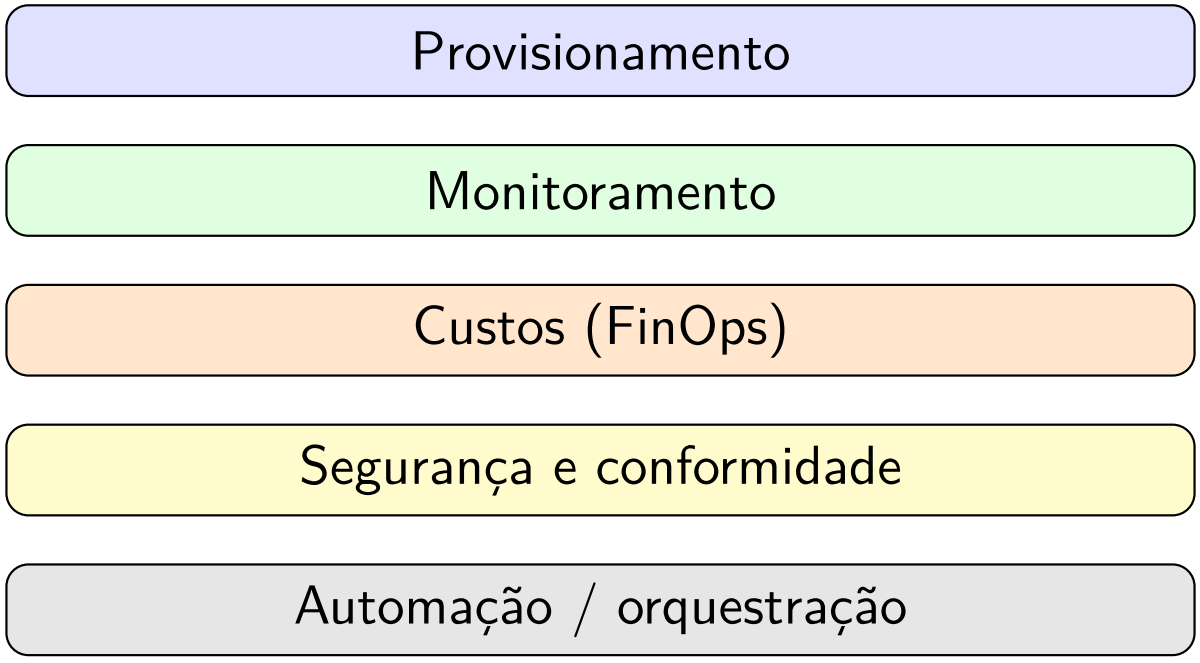
- Compreender o que é **gerenciamento de nuvem**.
- Conhecer plataformas e **ferramentas** de gestão.
- Entender **orquestração, automação e IaC**.
- Relacionar gerenciamento com **custos e governança**.

O que é Gerenciamento de Nuvem?

- Conjunto de práticas e ferramentas para **operar** recursos em nuvem.
- Abrange: provisionamento, monitoramento, custos, segurança e política.
- Objetivo: usar a nuvem de forma **eficiente e controlada**.
- Envolve **pessoas, processos e tecnologia**.

Camadas de Gerenciamento

- **Provisionamento:** criar recursos.
- **Monitoramento:** observar desempenho e saúde.
- **Custos (FinOps):** controlar gastos.
- **Segurança e conformidade:** políticas e auditoria.
- **Automação/orquestração:** reduzir trabalho manual.



Plataformas de Gerenciamento

- **AWS Management Console / Azure Portal / GCP Console**.
- **CLI e SDKs** para automação.
- Ferramentas **multi-cloud**: Terraform, Ansible, Pulumi.
- Painéis de monitoramento: CloudWatch, Azure Monitor.

Orquestração

- Coordena múltiplos recursos e serviços.
- Ex.: Kubernetes orquestra **contêineres**.
- Define **desejado** e reconcilia com o **estado real**.
- Base para **escalabilidade e resiliência**.

Automação e IaC

- **Infraestrutura como Código (IaC):** definir recursos em arquivos versionáveis.
- Ferramentas: **Terraform, CloudFormation, Ansible**.
- Reproduzível, auditável e reutilizável.
- Reduz erros manuais e acelera a entrega.

Monitoramento e Observabilidade

- **Métricas, logs e traces**.
- Alertas proativos sobre anomalias.
- Objetivo: **observabilidade** do sistema.
- Ex.: CloudWatch, Prometheus, Grafana.

FinOps — Gestão de Custos

- Unir **finanças + operações + tecnologia**.
- Visibilidade de gastos (tags, relatórios).
- Otimização: **rightsizing**, desligar ociosos, reservas.
- Cultura de responsabilidade sobre o custo.

Note

Gerenciar nuvem é **governar**: equilibrar agilidade, segurança, custo e conformidade. IaC e orquestração são peças-chave.

Resumo da Aula

- Gerenciamento cobre **provisionamento, monitoramento, custos e segurança**.
- **Orquestração** coordena recursos; **IaC** automatiza.
- **Monitoramento/observabilidade** garantem saúde e desempenho.
- **FinOps** controla e otimiza custos.

Atividade Rápida

- Cite **três funções** do gerenciamento de nuvem.
- Explique o que é **Infraestrutura como Código**.
- Dê um exemplo de **orquestração** (além de Kubernetes).

Referências

- Terraform — *Documentação*.
- FinOps Foundation.
- Documentação dos provedores de nuvem.

Aula 28 — Contêineres: Conceitos e VM $\times$ Contêiner

Objetivos da Aula

- Definir **contêineres** e sua origem.
- Comparar **contêineres** $\times$ **máquinas virtuais**.
- Compreender **isolamento** e **kernel compartilhado**.
- Reconhecer vantagens e casos de uso.

O que é um Contêiner?

- Pacote que reúne uma **aplicação e suas dependências**.
- Executa de forma **isolada** sobre o kernel do host.
- Leve, portátil e com **início rápido**.
- Não inclui um sistema operacional completo (compartilha o kernel).

Origem e Evolução

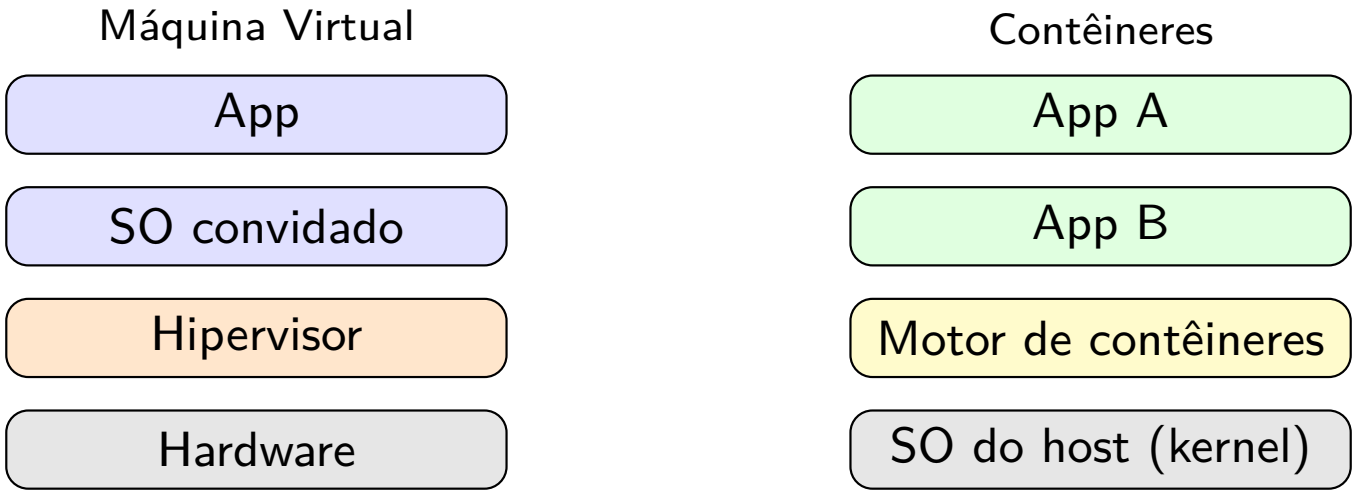
- **chroot** (1979) — isolamento de sistema de arquivos.
- **namespaces** e **cgroups** no Linux.
- **Docker** (2013) populariza os contêineres.
- **OCI** define padrões de imagens e runtimes.

Isolamento no Linux

- **Namespaces:** isolam processos, rede, PID, sistema de arquivos.
- **cgroups:** limitam CPU, memória e I/O.
- Resultado: processos “acham” que estão sós.
- Cada contêiner tem sua visão do sistema.

Contêiner × Máquina Virtual

Aspecto	VM	Contêiner
Kernel	Próprio	Compartilhado
Tamanho	GBs	MBs
Início	Minutos	Segundos
Isolamento	Forte (hardware)	Por processos
Densidade	Menor	Maior



Vantagens dos Contêineres

- **Leveza** e baixo consumo de recursos.
- **Portabilidade:** roda igual em dev, teste e produção.
- **Início rápido** e escalabilidade ágil.
- **Empacotamento** de dependências (fim do “na minha máquina funciona”).

Casos de Uso

- **Microsserviços**.
- **CI/CD** e ambientes reprodutíveis.
- **Aplicações portáveis** entre nuvens.
- **Escalabilidade** rápida e sob demanda.

Limitações

- **Isolamento** menos forte que VMs (compartilham kernel).
- Requer atenção à **segurança** da imagem e do host.
- Cargas com **kernel diferente** não rodam direto.
- Orquestração é recomendável em produção.

Note

Contêiner e VM **não são rivais**: muitas vezes o contêiner roda **dentro** de uma VM na nuvem.

Resumo da Aula

- Contêineres **empacotam a aplicação** e compartilham o kernel.
- São **mais leves e rápidos** que VMs.
- Isolamento via **namespaces e cgroups**.
- Ideais para **microsserviços, CI/CD e portabilidade**.

Atividade Rápida

- Compare **VM e contêiner** em três aspectos.
- Explique por que contêineres iniciam mais rápido.
- Cite dois **casos de uso** de contêineres.

Referências

- Docker — *Get Started*.
- Open Container Initiative (OCI).
- TANENBAUM, A.; BOS, H. *Sistemas Operacionais Modernos*.